

refers to the environment in which kernels execute. The context includes a set of devices, the memory accessible to those devices, and one or more command queues that schedule the execution of one or more kernels on the device. The OpenCL compiler uses the clang and LLVM technologies to compile the kernels and translate those instructions into optimised machine code targeted at the hardware on the host machine. The compiler first translates the OpenCL code to intermediate representation, and this code is further processed to generate the device code. With this approach, the same OpenCL program written once can be executed natively on various devices or machine architectures.

How to write an OpenCL program

The OpenCL program consists of host and device code (kernel). The host part executes on the CPU and the kernel part executes on the OpenCL computing devices (the CPU could also be one of them). The key mantra for writing effective OpenCL code and for optimal utilisation of the devices is to first identify the section of the application that can be parallelised. Given below are the 10 basic steps to write OpenCL programs for any device.

1. Look for the OpenCL platform. Given below is the API for detecting and getting IDs for OpenCL platforms:

```
cl_int clGetPlatformIDs (cl_uint num_entries,
                        cl_platform_id *platforms,
                        cl_uint *num_platforms)
```

2. Find out if any OpenCL devices are available and then initialise them. The API for finding the OpenCL devices is:

```
cl_int clGetDeviceIDs (cl_platform_id platform,
                      cl_device_type device_type,
                      cl_uint num_entries,
                      cl_device_id *devices,
                      cl_uint *num_devices)
```

Here, the device types could be either of the following:

```
CL_DEVICE_TYPE_GPU or CL_DEVICE_TYPE_CPU or
CL_DEVICE_TYPE_ACCELERATOR or CL_DEVICE_TYPE_CYSOTM or CL_
DEVICE_TYPE_ALL
```

3. If OpenCL devices are available and their initialisation is successful, then create a context and associate the devices. The following API creates an OpenCL context for the above devices:

```
cl_context clCreateContext (
const cl_context_properties *properties,
cl_uint num_devices,
const cl_device_id *devices,
void (CL_CALLBACK *pfn_notify)(const char *errinfo, const
void *private_info, size_t cb, void *user_data),
void *user_data, cl_int *errcode_ret)
```

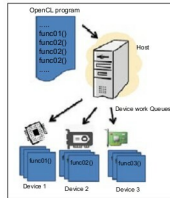


Figure 1: OpenCL work distribution

```
clCreateCommandQueueWithProperties (
cl_context context, cl_device_id device,
const cl_queue_properties *properties,
cl_int *errcode_ret)
```

5. Create and compile the device code. The following function creates a program object for a context (with associated devices) and loads the source code specified by the string:

```
cl_program clCreateProgramWithSource
(cl_context context,
cl_uint count, const char **strings,
const size_t *lengths, cl_int *errcode_ret)
```

The following function compiles and links a program executable from the program source for all the devices or to a specific device:

```
clBuildProgram (cl_program program,
cl_uint num_devices,
const cl_device_id *device_list,
const char *options,
void (CL_CALLBACK *pfn_notify)(cl_program program,
void *user_data, void *user_data))
```

6. Create the kernel. The following function helps to create the kernel object that will encapsulate the kernel function and the parameters/values to be used while executing those on a device:

```
cl_kernel clCreateKernel (cl_program program,
const char *kernel_name,
cl_int *errcode_ret)
```

7. Set the required arguments for the kernel. The following function helps to pass the arguments while starting the kernel on the device:

```
cl_int clSetKernelArg (cl_kernel kernel,
cl_uint arg_index,
size_t arg_size,
const void *arg_value)
```

4. Create a work queue (command queue) and associate the devices to the queue. The following function helps to create the command queue to enqueue the set of operations to host or device:

```
cl_command_queue
```